Architecture & Setup Documentation

Overview

This project implements a simple **event-driven ELT pipeline**:

```
AWS Lambda  →  Amazon S3  →  Snowflake (RAW → STAGE → FINAL)
```

Lambda generates data and drops it into S3 as JSON files, partitioned by date. Snowflake then reads those files through a **Storage Integration** (a secure, keyless connection using IAM roles) and progressively refines the data through three layers.

Why three layers (RAW → STAGE → FINAL)?

Layer	What it holds	Why it exists
RAW	Data exactly as it arrived (raw JSON)	Acts as the source of truth. If a downstream bug corrupts STAGE/FINAL, you can always rebuild from RAW without re-fetching from the original source.
STAGE	Cleaned, typed, flattened columns	A safe working layer — JSON fields are extracted into proper columns (STRING, FLOAT, NUMBER, etc.), ready for business logic.
FINAL	Aggregated, business-ready tables	What reporting tools/dashboards query directly. Pre-aggregated so queries are fast and simple.

Step-by-Step Setup

1. AWS Side

1.1 Create an S3 bucket

```
aws s3 mb s3://your-bucket-name --region us-east-1
```

1.2 Create an IAM role for Lambda with an inline policy allowing `s3:PutObject` scoped to `raw/*` in your bucket.

1.3 Deploy the Lambda function (see `lambda/lambda_function.py`) — it generates 50 fake sales records and writes them to `s3://your-bucket-name/raw/yyyy/mm/dd/`.

1.4 (Optional) Schedule it with EventBridge so it runs automatically every N minutes:

```
aws events put-rule --name run-fake-data-generator --schedule-expression "rate(5 minutes)"
```

2. Snowflake ↔ AWS Connection (Storage Integration)

This is the most important — and most commonly misconfigured — part.

1. In Snowflake, run `01_storage_integration.sql` to create the `STORAGE INTEGRATION`.
2. Run `DESC INTEGRATION s3_int;` and copy two values:
   - `STORAGE_AWS_IAM_USER_ARN`
   - `STORAGE_AWS_EXTERNAL_ID`
3. In AWS, create an IAM role with this **trust policy** (fill in the two values above):

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": { "AWS": "<STORAGE_AWS_IAM_USER_ARN>" },
      "Action": "sts:AssumeRole",
      "Condition": {
        "StringEquals": { "sts:ExternalId": "<STORAGE_AWS_EXTERNAL_ID>" }
      }
    }
  ]
}
```

4. Attach a **permissions policy** to the same role, granting read access to your bucket:

```
{
  "Version": "2012-10-17",
  "Statement": [
    { "Effect": "Allow", "Action": ["s3:GetObject"], "Resource": "arn:aws:s3:::your-bucket-name/raw/*" },
    { "Effect": "Allow", "Action": ["s3:ListBucket"], "Resource": "arn:aws:s3:::your-bucket-name",
      "Condition": { "StringLike": { "s3:prefix": ["raw/*"] } } }
  ]
}
```

5. Back in Snowflake, run `LIST @s3_stage;` — if your S3 files show up, the connection is working.

3. Load Data (RAW → STAGE → FINAL)

Run the SQL scripts in order:

```
02_raw.sql     → loads raw JSON from S3 into the RAW table
03_stage.sql   → flattens JSON into typed columns
04_final.sql   → aggregates into business-ready summary tables
```

Common Issues & Fixes

Symptom	Likely Cause	Fix
`LIST @s3_stage;` returns access denied	Trust policy or permission policy misconfigured	Double-check the `STORAGE_AWS_IAM_USER_ARN` / `ExternalId` match exactly, and bucket name in permission policy is correct
“Database not found” errors	No database/schema selected in session	Run `USE DATABASE EVENT_PIPELINE; USE SCHEMA RAW;` first
Columns come back `NULL` after flattening	Data already flattened by `STRIP_OUTER_ARRAY`, don’t apply `LATERAL FLATTEN` again	Query `raw_json:field::TYPE` directly, no `LATERAL FLATTEN` needed
Lambda runs but no file in S3	Wrong bucket name in code, or looking in wrong S3 folder path	Verify with `aws s3 ls s3://your-bucket-name/raw/ --recursive`

Future Improvements

- **Snowpipe**: replace manual `COPY INTO` with continuous, event-driven ingestion — S3 notifies Snowflake automatically when a new file lands.
- **Streams & Tasks**: automate STAGE → FINAL refresh whenever RAW gets new rows.
- **dbt**: for more complex, versioned transformation logic as the project grows.
- **Monitoring**: CloudWatch alarms on Lambda errors; scheduled checks on Snowflake `TASK_HISTORY`.